Seica
Test Solutions

www.seica.com

NEXT SERIES

VIVA SOFTWARE

COMPLEX DEVICES

VIP
VIVA INTEGRATED PLATFORM

VIVA SOFTWARE
COMPLEX DEVICES

PUBLICATION ISSUED BY:

Via Kennedy, 24 - 10019 STRAMBINO (TO)
Tel.: +39 0125 63 68 11
Fax.: +39 0125 63 68 99
e-mail: seica@seica.com
Web site: http://www.seica.com

Copyright © 2025
All rights reserved

Microsoft Windows is a registered trademark of Microsoft Corporation

Document P/N: MA-VI-VIVADCEN-04
Software release: VIVA 15.0.0.x'
Date of issue: 2025-10-27

Seica S.p.A. * reserves the right to change the technical specifications without notice.

* The names Seica or Seica S.p.A. reported in this manual also refer to Seica Inc., Seica Sarl, Seica China, affiliates, subsidiaries and divisions Seica.

TRANSLATION OF THE ORIGINAL INSTRUCTIONS

[Limitazione di responsabilità]. Seica S.p.A. garantisce che le informazioni disponibili nella versione italiana di questo documento sono accurate ed aggiornate. Seica S.p.A. non si assume alcuna responsabilità per eventuali errori di trascrizione, tipografici od omissioni. In caso di discrepanze nella traduzione, prevale la versione italiana del documento.

[Disclaimer]. Seica S.p.A. guarantees that the information available in the Italian version of this document are correct and updated. Seica S.p.A. declines any liability for any clerical, typographical errors or omissions. In case of discrepancies in the translation, the Italian version of the document shall prevail.

CONTENTS

CONTENTS 1-1
LIST OF FIGURES 1-2
LIST OF TABLES 1-2
1. GENERAL 1-1
1.1 TRANSLATION OF THE ORIGINAL INSTRUCTIONS 1-2
1.2 SEICA CUSTOMER CARE SERVICE 1-2
1.2.1 PHONE NUMBER AND E-MAIL 1-2
1.2.2 TICKETING SERVICE 1-2
1.2.3 REMOTE CONNECTION 1-3
1.2.4 INTERNET INFORMATION 1-3
1.2.5 SYMBOLS AND SYNTAX 1-3
2. COMPLEX DEVICES IN VIVA ENVIRONMENT 2-1
2.1 VIVA – REFERENCE DATABASE, COMPONENTS AND COMPLEX DEVICES 2-2
3. FUNCTIONS AND COMPLEX DEVICES 3-1
3.1 SIMPLE DEVICES AND COMPLEX DEVICES 3-2
3.2 COMPLEX DEVICE MADE UP BY DISCRETE COMPONENTS 3-3
3.2.1 ADD FUNCTIONS 3-4
3.2.2 ADD A COMPLEX DEVICE 3-6
3.2.2.1 ASSIGN A DEVICE TO A COMPLEX DEVICE 3-9
3.2.2.2 ASSIGN A COMPONENT TO A COMPLEX DEVICE 3-11
3.2.2.3 ASSIGN A FUNCTION TO A COMPONENT 3-13
3.2.2.4 ASSIGN A MACRO TO A COMPONENT 3-14
3.2.3 CHANGE THE MACROS BELONGING TO A FUNCTION 3-16
3.3 COMPLEX DEVICE CONSISTING OF ICS 3-17
3.3.1 ADD FUNCTIONS 3-21
3.3.2 ADD A COMPLEX DEVICE 3-22
3.3.3 ASSIGN COMPONENTS TO COMPLEX DEVICES 3-25
3.3.4 COMPLEX DEVICE MADE UP BY SEVERAL MACROS 3-27

LIST OF FIGURES[1]

Fig. 2-1 VIVA SOFTWARE: Program structure and association Function-Components 2-3
Fig. 2-2 VIVA SOFTWARE: Complex Components, New Device and Complex Device 2-8
Fig. 3-1 VIVA SOFTWARE: Devices and Components list for two components of a complex device 3-3
Fig. 3-2 VIVA SOFTWARE: The "Complex Component Wizard" dialog window 3-6
Fig. 3-3 VIVA SOFTWARE: "Add Complex Component" from Programming Environment 3-7
Fig. 3-4 VIVA SOFTWARE: Complex device and VIVA models for Operational Amplifier 3-18
Fig. 3-5 VIVA SOFTWARE: Device and Component list after "import Data" 3-19
Fig. 3-6 VIVA SOFTWARE: "OpAmp" and "Digital" functions view from the Debug Environment 3-19
Fig. 3-7 VIVA SOFTWARE: "Assign Function" tables 3-20
Fig. 3-8 VIVA SOFTWARE: "Debug Environment" of the VIVA Software 3-20
Fig. 3-9 VIVA SOFTWARE: Adding functions in the Programming Environment 3-21
Fig. 3-10 VIVA SOFTWARE: Complex device "Operational Amplifier" used as Amplifier 3-23
Fig. 3-11 VIVA SOFTWARE: Complex device "Operational Amplifier" used as Comparator 3-23
Fig. 3-12 VIVA SOFTWARE: Complex device "Operational Amplifier" used as Voltage Follower 3-24
Fig. 3-13 VIVA SOFTWARE: "Assign Function" Component table 3-25
Fig. 3-14 VIVA SOFTWARE: Device and Component list after "Assign Function" 3-25
Fig. 3-15 VIVA SOFTWARE: "Debug Environment" 3-26

LIST OF TABLES

Tab. 2-1 VIVA SOFTWARE: Seica CAD data file (*.NOD) 2-6
Tab. 2-2 VIVA SOFTWARE: Seica CAD data file (*.PAR) 2-6

[1] The figures included in this manual may not correspond to the last product release. Their aim is to support the topics developed in the manual.

1.GENERAL

This manual provides an overview of the VIVA Software. It is intended for Expert Users/Programmers/Administrators of test systems engineered and manufactured by Seica S.p.A.
This manual describes the Complex Devices, also designated as “Complex Components” and it is intended for “Developer” or “Administrator” user categories of the VIVA Software.

1.1 TRANSLATION OF THE ORIGINAL INSTRUCTIONS

This document is a translation of the original instructions supplied in delivery.

1.2 SEICA CUSTOMER CARE SERVICE

Seica S.p.A. provides technical support to its customers through different channels.

Before contacting the technical support, make sure the following information is available:

- Type or hardware system
- System Serial Number
- Software release installed

The technical support can be contacted via:

- Phone number and e-mail
- Ticketing service
- Remote connection

The technical support service, if present and specified in the relevant purchase order or in the specific use of the software release.

1.2.1 PHONE NUMBER AND E-MAIL

S.E.I.C.A. S.p.A. provides technical support via phone/E-mail during working hours to solve possible problems which may occur when using the system to the following number:

Phone Number	0039-125-636811
E-mail	service@seica.com

1.2.2 TICKETING SERVICE

The ticketing service can be access through the dedicated online platform.

The service is conceived to provide technical support in an efficient and tracked manner.
The user submits its request via the platform, and each ticket is assigned a unique ID number to track and handle the requests.

Online platform:

https://seicagroup.freshdesk.com/support/login

Please refer to the document "Fresh Desk Guide" supplied with the documentation folder, for further information concerning the utilization of the platform and ticket submission.

1.2.3 REMOTE CONNECTION

The SEICA technical support technician contacted via phone/e-mail support or ticketing service, will evaluate on a case by case basis if a remote connection must be used, while providing the information to activate the connection.

1.2.4 INTERNET INFORMATION

Updated information concerning products, technologies and applications are available on the website: http://www.seica.com

1.2.5 SYMBOLS AND SYNTAX

In this manual, the following symbols and syntax have been used.

WARNING: Indicates a potential hazard for the operator or the system as a possible result of the action described in the warning.

NOTE: Indicates a note, a recommendation or a point where the reader should focus more carefully.

For ease of reference, the text in the manual features crossed references to sections/figures as shown in these two examples:

1.2.3REMOTE CONNECTION

Refer to Fig. 3-3

Refer to Tab. 2-1

By holding the CTRL key and hovering the mouse on the text, the symbol of a hand is prompted , by clicking on the text or on the section number/figure, you will surf to the relevant text/figure.

[Key 1]+ **[Key 2]** = Press Key 1 and Key 2 simultaneously.
[Key 1], **[Key 2]** = Press Key 1, release it, and then press Key 2.

The files are indicated with the following syntax:

File.XML
Where "File" indicates the filename and ".XML" is the file extension.

2.COMPLEX DEVICES IN VIVA ENVIRONMENT

The **Programming Environment** is used to add new functions or to modify the existing ones in regards to the test program creation with the VIVA software. In this environment it is possible to:

- Define the link between a kind of component and the functions executable on that component through the VIVA management software
- Add new functions to those provided with the installation
- Define the macros that make up a specific function
- Define the labels that make up a specific macro
- Define the executive code for a specific label of a specific macro; the executive code consists in recalling the system instruments with the associated operation (o method) which the instrument can perform.

This Manual provides an overview and operating instructions concerning the Complex Devices (also designated as "Complex Components"). The instructions and information supplied in this manual are intended for Expert Users/Programmers/Administrators of the VIVA Software.

2.1 VIVA – REFERENCE DATABASE, COMPONENTS AND COMPLEX DEVICES

The Fig. 2-1 summarizes the activities available with the VIVA Software for test program development and debug purposes. The VIVA Software provides a guided procedure (designated as "Wizard Environment") to walk the User through the steps required to create, debug and execute a test program for an electronic board.

For an overview of the VIVA Software environment, press the from the VIVA Home page.

Or refer to the relevant document in the documentation file supplied in delivery.

The Programming Environment allows to define the functions required in the "Function & Macro database" which, combined with the structures available in "Board CAD data", are used to create test programs and test UUTs made up by **components** grouped in **devices**.

A **function** is associated to each device, and is used to test the components belonging to that specific device.

The functions are made up by one or more **macros**, which form the code executed to check the components.

It is possible to parameterize the macros according to the features of the component to be checked (value of the scale to be measured, accepted tolerances, identification of the component pins, etc.).

*NOTE: The macros are made up by groups of code designated as: **labels**. The labels can be executed either in sequence or with a flow control based on the instruction: "Jump: <condition> To: <label>", executable at the end of each label.*

The labels are made up by instructions which manage the **tools** available in the VIVA Environment.

The tools are **software objects** aimed to manage the resources[2] in the VIVA Environment. The tools are identified by a name and can be accessed through an interface made up by methods identifying the

[2] The term "resources" does not merely identify the Hardware instruments used to manage the measurements, but also the Software instruments implementing the Native VIVA Language.

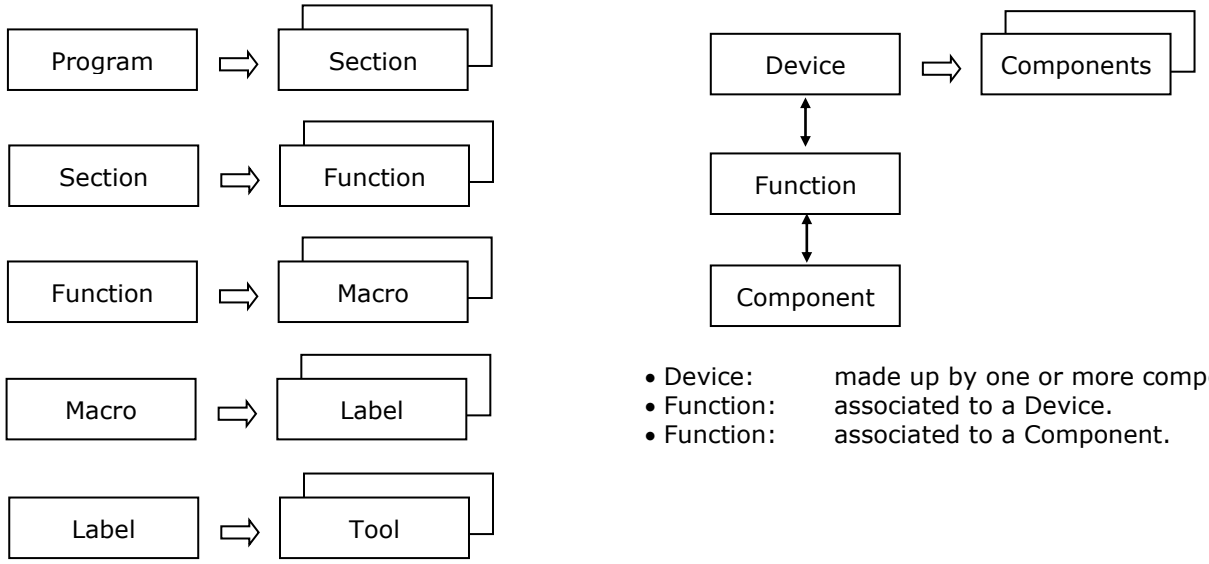
operations required to the software object; instead, it is not possible to access the operations executed by the software object. The instructions for tool management are identified by the couple <**Tool.method**>

The method identifying a function to access the tool, schedules a variable number of parameters according to the method itself. A generic instruction to manage a tool will have the following format:
<**Name_Tool.Name_Method(par*1*, par*2*, ... Par*n*)**>

In a label, the control of the instructions flow is performed by the instructions of the **Native VIVA Language**, which can be created through a graphic interface available in the Programming Environment.

For every function, the attribute **Section** allows to group one or more functions with a section name which can be defined by the user. The section name will appear in the Debug Environment to identify the set of components checked with the same macros. In the Debug Environment the name of the function does not appear; it is used only in the Create Program phase to associate the appropriate macros to the components to be checked.

The drawings in Fig. 2-1 show the links of the different structures related to a program executable in the VIVA Environment. The Programming Environment allows to define the composition and links of the structures.



- Program: made up by one or more Sections.
- Section: made up by one or more Functions.
- Function: made up by one or more Macros.
- Macro: made up by one or more Label.
- Label: made up by one or more Tool.

- Device: made up by one or more components.
- Function: associated to a Device.
- Function: associated to a Component.

Fig. 2-1 VIVA SOFTWARE: Program structure and association Function-Components

Some paragraphs of this section will provide examples concerning the following actions:

- Delete and add macros to a function.
- Create a new function.
- Add a new component.
- Assign function(s) to a new component.

As a preable to the following sections a general overview of the system macros context and teir use in VIVA programs is provided.

External CAD data files

These are the CAD data of the board under test (hereinafter referred to as "UUT") containing all of the required information to locate the test points and the values to be verified. As an example, you will find the description of the fields used in the CAD Seica format, which schedules two kinds of records: one to identify the testpoints physically and the other to identify the components that form the board.

Below, the fields of the records containing information about the nodes of a board:

SIGNAL	Identifies the name of the signal present on the node, it is a reference to the electrical schemes of the board.
COMPONENT	Identifies the name of the component associated to the node. Together with the "**PIN"** attribute, it identifies univocally the name of the node.
PIN	Identifies the pin number of the component associated to the node. Together with the attribute "COMPONENT", it identifies univocally the node name.
X	Identifies, in the space X,Y containing the board, the x-axis of the node. Together with the attribute "Y", it identifies the node location.
Y	Identifies, in the space X,Y containing the board, the y-axis of the node. Together with the attribute "X" it identifies the node location.
PAD	Defines the size and shape of the PAD associated to the node.
POS	Indicates the board side where the node is located (T for Top, B for Bottom O for bOth).
TECN	Identifies the technology of the component mounted on the node (T for Trough hole and S for SMD).
TEST	Identifies the scheduled elaboration on the node (Y for Test and N for Don't test).
CHANNEL	Node id number (also called Channel) defined by the system. This field is not relevant in the external CAD files and is filled-in during the import data phase into the Board CAD data base

All the fields (10) must be filled in! In case of missing information, the field **must** be filled in with the ASCII character *zero* (0).

Here below are listed the fields of records containing information about board components:

COMPONENT	Identifies the name of the component associated to the node. It is the same name present in the field "Component" of the record identifying the node on the board.
DEVICE[3]	Identifies a class (or kind) of components. In the VIVA environment, a function is associated to each "DEVICE" in order to test the components belonging to "DEVICE".
VALUE	Identifies the parameter value of the component to be tested.
UNIT	Identifies the measure unit of the parameter to be verified.
TOL+	Identifies the positive tolerance of the value to be verified.
TOL-	Identifies the negative tolerance of the value to be verified.
NRPIN	Identifies the whole pin number of the component.
MACRO/FUNCTION	Identifies the macro or the function associated to the component. If the field is not filled in, the function associated to the component is the one indicated in "Function & Macro data base"

The first seven fields must be filled in! In case of missing information, the field **must** be filled in with the ASCII character *zero* (0). Only the field "MACRO/FUNCTION" can be left empty.

Board CAD data

These details are referred to the board to be tested, containing the information needed to locate the nodes, and the values that have to be verified. It's the same kind of information that can be found in the "External CAD data files" but recorded in the board data base. The introduction can be done through the "Import CAD data" function or the "Import Manual Data" function.

Function & Macro database

These details are referred to the definition of functions, associated components, associated macros and the value of macro parameters. The Information included in this group of data, together with those included in the "Board CAD data base" are used to create the testing program identified by the series of data "Program database".

Program database

These details are referred to the test program; they are located in the system database. These contain data used during the program execution. The Data have been generated in the "Create Program" phase, along with the changes made by the user (e.g. in the Debug Environment) or by executing the program in "Autodebug" or "Autolearn" modes.

External CAD data files & XML Program data file

This group contains two kinds of details: those related to board CAD data and those related the program generated and verified with the information fixed at the "Export Data" execution.

- The details related to board CAD data are stored in CAD files providing the information needed to locate the nodes on the board, and the values that must be tested. The kind of details are the same as "External CAD data files" details, and can be used to execute the "Import CAD Data" function.
- Details related to the created program consist of a program description ("*.XML" format). The program "*.XML" format, can be used during "Create Program" phase, selecting the button: "Update Program …" instead of the button: "Create Program !".

[3] In the "Board CAD database" can be found the item "Complex devices", which identify devices made up by one or more parts (SUB*i*). To each part (SUB*i*) are associated the pins of the device and the macro used to test the part. These "Complex Devices" are created and introduced only in "Board CAD database" and are not introduced in the "External CAD data files".

The Tab. 2-1 and Tab. 2-2 show and outline a sample record of the “SeicaCAD data files”.

Tab. 2-1 VIVA SOFTWARE: Seica CAD data file (*.NOD)

SIGNAL	COMPONENT	PIN	X	Y	PAD	POS	TECN	TEST	CHANNEL
……	……	..	……..	……..	……..	.	.	.	…
“ANODEDI1”	“DI1”	1	170,000	185,000	R39H23	T	T	Y	573
“ANODEDI2”	“DI2”	1	170,000	187,000	R39H23	T	T	Y	574
“CATODEDI1”	“DI1”	2	172,000	185,000	R39H23	T	T	Y	575
“CATODEDI2”	“DI2”	2	172,000	187,000	R39H23	T	T	Y	576
"ACDC1_AC"	"ACDC1"	1	175.000	187.000	R31H23	O	T	Y	652
"ACDC1_DCA"	"ACDC1"	2	175.000	186.000	R31H23	O	T	Y	653
"ACDC1_DCB"	"ACDC1"	3	175.000	185.000	R31H23	O	T	Y	653
"ACDC1_DC+"	"ACDC1"	4	175.000	184.000	R31H23	O	T	Y	654
"ACDC2_AC"	"ACDC2"	1	177.000	187.000	R31H23	O	T	Y	655
"ACDC2_DCA"	"ACDC2"	2	177.000	186.000	R31H23	O	T	Y	656
"ACDC2_DCB"	"ACDC2"	3	177.000	185.000	R31H23	O	T	Y	656
"ACDC2_DC+"	"ACDC2"	4	177.000	184.000	R31H23	O	T	Y	657
……	……	..	……..	……..	……..	.	.	.	…

Tab. 2-2 VIVA SOFTWARE: Seica CAD data file (*.PAR)

COMPONENT	DEVICE		VALUE	UNIT	TOL+	TOL-	NRPIN		MACRO/Function
…….	……		……		……	……	……	……	……
“DI1”	“Diodes”		0.30	V	0.00	0.00	2		
“DI2”	“Diodes”		0.40	V	0.00	0.00	2		
"ACDC1""	DiodeResistors"	0.70	V	0.00	0.00	4			
"ACDC2""	DiodeResistors"	0.70	V	0.00	0.00	4			
"RL1"	"DRelais"		0.00	V	0.00	0.00	4		
"RL2"	"DRelais"		0.00	V	0.00	0.00	4		
…….	……	……		……..	……	……	……		……

According to the records described in the previous tables, the “Import CAD data” phase associates the function “Diode” to devices kind “Diodes” and “DiodeResistors”.

In the “Assign Function” dialog window of tab “Create Program” it is possible to view:

Device	Function	Components
……..	……..	…….
DIODERESISTORS	DIODE	ACDC1,ACDC2
DIODES	DIODE	DI1,DI2
……..	……..	…….

In the Programming Environment, the item “Patterns” of the “Diode” function contains the instruction[4] “DIODE*” which defines the association of function “Diode” to all devices whose name begins with string “DIODE”. (hence also the devices: “Diodes” and “DiodeResistors”).

In the Debug Environment, it is possible to view the structure of the “Diode” function:

```
[-] Functions
 +----[-] Diode
          +----[-] Associated Macros
          |        +--DIODE
          +----[-] Patterns
                   +--##CR*
                   +--##D*
                   +--##GR*
                   +--##LD*
                   +--##LED*
                   +--DIODE*
```

[4] For further information about the Programming Environment, please refer to the VIVA on-line documentation and to the dedicated manual referenced in the Bibliography.

According to the reference CAD data, in the Debug Environment it is possible to view the structure of the "Diode" section.

When the program is executed, the "Diode" section identifies the execution of the "Diode" function on all components belonging to the devices to whom the "Diode" function is associated.

The execution of the "Diode" function on a component identifies the execution of the macros which make up the "Diode" function. (In this case the "Diode" function is made up by a single macro named "DIODE").

Test sections in the Debug Environment:

```
[-] DIODE
  +----[-] DI1
  |         +--DIODE
  +----[-] DI2
  |         +--DIODE
  +----[-] ACDC1
  |         +--DIODE
  +----[-] ACDC2
            +--DIODE
```

To avoid associating the "Diode" function to the "DIODERESISTORS" devices when creating the program, it is possible to assign the device a name not included in those considered in the directory "Patterns" of the "Diode" function present in the Programming Environment.

For example:

COMPONENT	DEVICE		VALUE	UNIT	TOL+	TOL-	NRPIN	MACRO/Function
……..	…….		……	….	……..	……..	……	……
"DI1"	"Diodes"		0.30	V	0.00	0.00	2	
"DI2"	"Diodes"		0.40	V	0.00	0.00	2	
"ACDC1""***XDiodeResistors***"		0.70	V	0.00	0.00	4		
"ACDC2""***XDiodeResistors***"		0.70	V	0.00	0.00	4		
"RL1"	"DRelais"		0.00	V	0.00	0.00	4	
"RL2"	"DRelais"		0.00	V	0.00	0.00	4	
……	…….		…….	….	……..	……..	……	……

During the "Import Data" phase the "Assign Function" dialog window allows to associate a function to the "XDiodeResistors" device. If no association is made, the "Assign Function" dialog window in the "Create Program" page of the Wizard Environment will show:

Device	Function	Components
……..	…….	……
XDIODERESISTORS	UNKNOWN	CDC1,ACDC2
ZENERS	ZENER	ZE1,ZE2
……..	……	……

In the Debug Environment, the sections of the grid on the left-hand side of the page, show that ACDC1 and ACDC2 components, belonging to "XDIODERESISTORS" device, are included in section UNKNOWN, without a macro associated. Therefore, these components will not be tested.

```
[-] UNKNOWN
  +---- ACDC1
  +---- ACDC2
  +----
```

The "DiodeResistors" device, described in Tab. 2-2, and subsequently renamed as "XDiodeResistors", is an example of a device with no specific test function in the Programming Environment.

The VIVA Software provides two possible solutions to test this component:

1. Define a new function, made up with the available MACROS which refer to device templates. By associating this function to the device, it is then possible to generate a test program for the components belonging to the device. Since the macros refer to sample devices, in the Debug Environment it is necessary to modify the reference to component pins not corresponding to the actual component. (e.g. for the diode, the macro schedules that pin 1 is associated to the anode and pin 2 to the cathode; the reference pin for resistance are pins 1 and 2; the resistance of the actual component is connected to pins 3 and 4. Therefore, after the automatic program generation, it is necessary to modify the program to match the components description included in the CAD data files.
2. Define a complex device made up by a sufficient number of sub-devices which can be checked using the functions included in the "Function & Macro database". So, it is possible to generate a test program which does not require subsequent modifications to align the sample component and the real component description.

With solution number 2., the complex device is defined in the "Function & Macro database". It is then possible to automatically execute all the activities scheduled to create a test program for the real UUT. In the Debug Environment it is possible to execute the verify, autolearn and autodebug steps without need of alignment between the default UUT description and the description of the real UUT.

The drawings in Fig. 2-2 depict the pinout of the complex components: ACDC1 and ACDC2 and the two possible classifications with a new devide (XDiodeResistors) or with a complex device (ACDC).

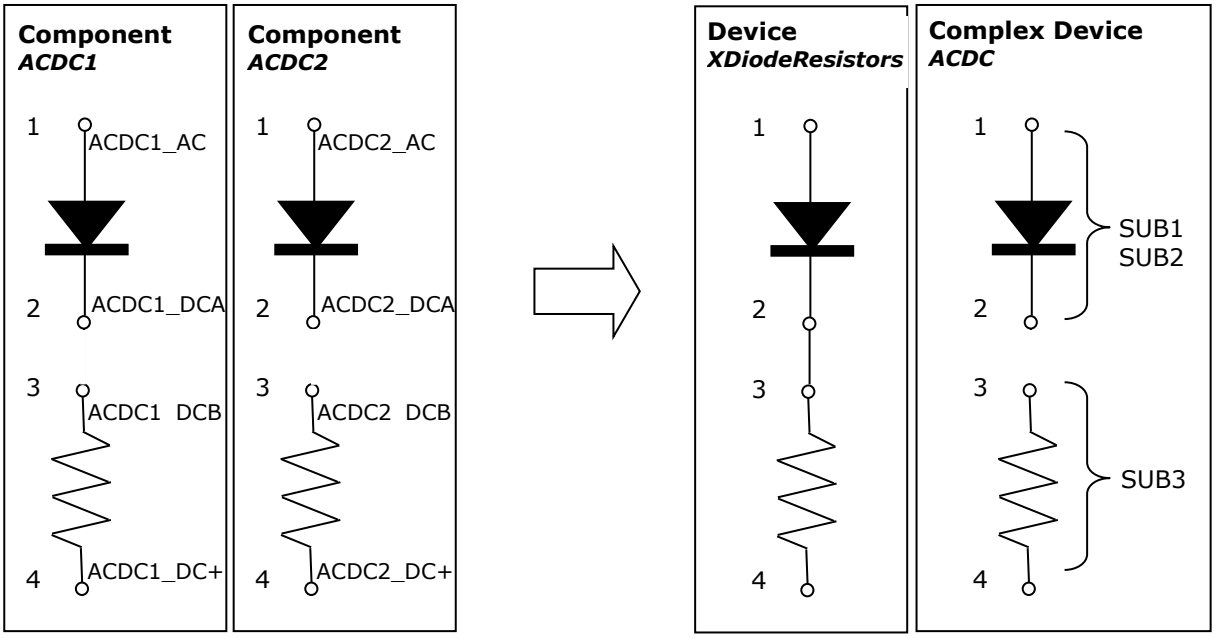


Fig. 2-2 VIVA SOFTWARE: Complex Components, New Device and Complex Device

After the general information provided in this section to help understanding the context of complex devices generation, the next section describes the activities to be executed in the Programming Environment to generate such devices. For further information, please refer to the VIVA Software on line documentation and to the relevant documentation supplied in delivery and included in the Seica Product documentation folder.

3.FUNCTIONS AND COMPLEX DEVICES

The directories or patterns available in the Programming Enviroment are a way to associate functions to components and devices present in the CAD data of the UUT.

Another way to associate functions to components and devices is to use the "Complex Devices". (In the VIVA GUI, these "Complex Devices"are also designated as "Complex Components")[5].

- A complex device identifies a kind of component which may be divided into one or more sub components: SUB1, SUB2, …SUBn.
- Each sub component is defined by:

  1. The number of pins used
  2. For each pin used: by the pin identification number related to the complex device
  3. By the function associated to check the specific sub component. The function must be included in the list of functions available in the Programming Environment.

- In the VIVA GUI, a function designated as "<Complex>" is associated to the complex device to distinguish this kind of functions from those available in the Programming Environment.
- In the Debug Environment the name of the section associated to the complex device corresponds to the name of the function associated to the first "sub device" (SUB*1*) of the complex device.

[5] Specifically: a component belongs to a class or kind of components designated as "device". A complex device belongs to a class or kind of components designated as "complex devices".

3.1 SIMPLE DEVICES AND COMPLEX DEVICES

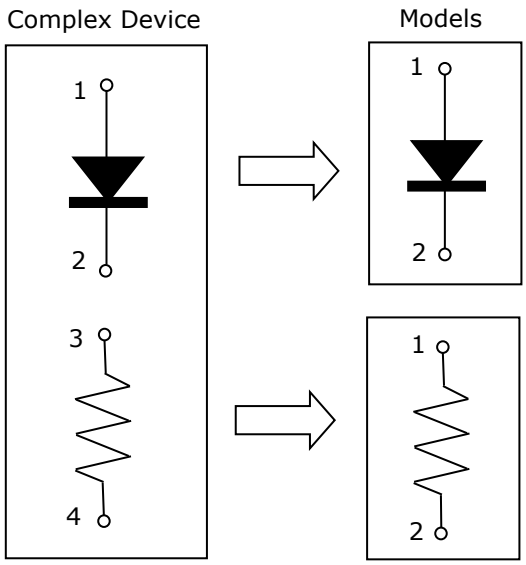
1. An existing function in the VIVA Software schedules the execution of one or more system macros to check the simple or theoretical devices described in the VIVA Software.
2. The simple devices are theoretical devices with a given pin numbering. In other words, the simple devices may be considered as samples which may require mapping in order to be applied to physical models so that the VIVA Software may interpret them.
3. For testing, the physical devices may have a pin numbering other than that of the simple device stored.
4. With the complex devices it is possible to perform the mapping between the numbering of the simple and the physical device.
5. **It is necessary to assign to every pin of the simple device the number of the corresponding pin on the physical device**. It is important to perform this operation, even though the associated function uses a pin number lower than that defined for the theoretical device.
6. The physical devices may be made up by one or more sub-devices and the mapping between theoretical component and physical component may be defined for each sub-device.

With the VIVA Software, a complex device is made up by one or more sub-devices, and it is possible to associate a test function to each sub device. The "Complex Component Wizard" dialog window allows to define the mapping between simple devices (theoretical devices) and sub-devices (physical devices), besides, it allows to associate the function used to check the sub-device.

The following paragraphs refer two examples of use of complex devices:

- The first example is referred to the management of an analog complex device made up by sub-devices which can be tested using functions made up by macros available in the models stored in the Programming Environment.
- The second example is referred to the management of a digital complex device where the sub-devices are all the same. Also in this case, it is possible to execute the check through functions made up by the macros available in the models stored in Programming Environment.

3.2 COMPLEX DEVICE MADE UP BY DISCRETE COMPONENTS



The example used is that of the ACDC complex device reproduced in Fig. 2-2, to whom the two components "ACDC1" and "ACDC2" are associated. These two components are described with the two Seica CAD data files reported in Tab. 2-1 and Tab. 2-2.

To check the complex device, it is necessary to define the function used to check the "diode" and the "resistor" sub devices (this solves the pin mapping which, for the resistor type component is connected to pins 3 and 4 on the complex device, while on the sample device it is connected to pins 1 and 2).

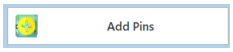
The basic steps are the following:

1. Add the functions required to check all the sub-devices which form the complex device. (FACDC, FACDC_DIODE and FACDC_RESISTOR).
2. Add the complex device which describes all the sub-devices. (DACDC with SUB1, SUB2 and SUB3).
3. Assign a device or a component to a complex device.

This implies the following changes in the "board CAD data":

1. It is necessary to add to the component list: the original component and as many components as many subcomponents (*SUBi*) make up the complex component.
2. It is necessary to add to the device list:
   - A device to identify the complex device
   - As many devices as many are the functions associated to the constituent subcomponents of the complex device.

Below it is provided the list of the additional components and devices in the UUT database upon assigning two components (ACDC1 and ACDC2) to a complex device (DACDC) consisting of three sub-devices. Each sub-device is tested with the related functions: FACDC, FACDC_DIODE and FACDC_RESISTOR.

It is possible to view the list of components and devices included in "Board CAD data" using the "Add Items Wizard" dialog window, pressing the button 

in the "Edit Data" page of the Wizard Environment.

<u>Device board list</u>

```
.....
DACDC                ( FACDC )
.....
FACDC                ( FACDC)
FACDC_DIODE          ( FACDC_DIODE )
FACDC_RESISTOR       ( FACDC_RESISTOR )
 .....
```

<u>Component board list</u>

```
.....
ACDC1
ACDC1#1
ACDC1#2
ACDC1#3
ACDC2
ACDC2#1
ACDC2#2
ACDC2#3
.....
```

Fig. 3-1 VIVA SOFTWARE: Devices and Components list for two components of a complex device

3.2.1 ADD FUNCTIONS

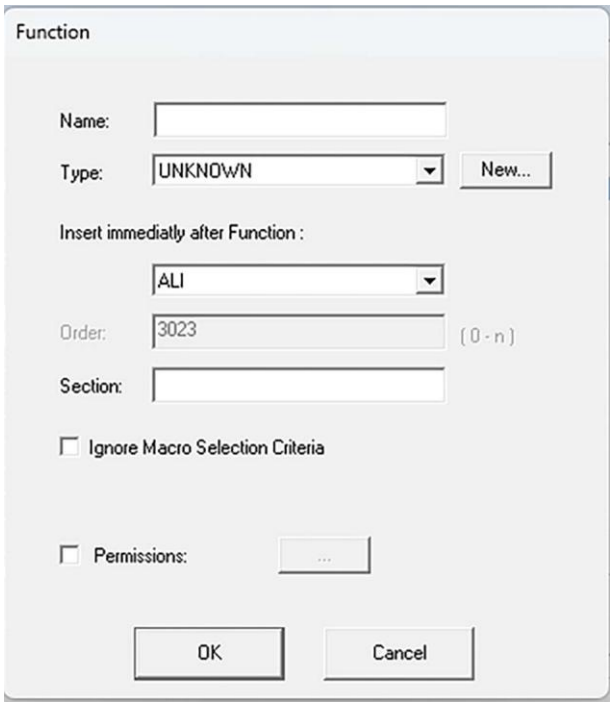
This section describes the steps required to add functions.

- Function: "FACDC" Does not contain any system macro.
- Function: "ACDC_RESISTOR" Contains the "RESISTOR" system macro.
- Function: "ACDC_DIODE" Contains the "DIODE" and "DIODE_INV" system macros.

1. Double-click the icon on the Windows desktop to launch the VIVA Software.
2. Login to the VIVA software as "Administrator" or "Developer" user category
3. In the main page of the VIVA software, press the button to open the right-hand side toolbar.
4. In the Utilities section, press the FAT Markers_ori Version 1 to enter the Program Manager Utility.
5. In the Program Manager Utility, use the search bar or the side scroll bar until you find the and enter the Programming Environment

<u>In the tab "General" of the Programming Environment:</u>

6. Select the Add Function button.



7. In the field "Name" of the "Function" dialog box, type "FACDC"; flag the checkbox Ignore Macro Selection Criteria and press the OK button.

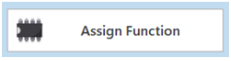
8. Select the button Add Function
9. In the field "Name" of the "Function" dialog box, type: "ACDC_RESISTOR" flag the checkbox Ignore Macro Selection Criteria and press the OK button.
10. Select the function: ACDC_RESISTOR Select the item "Associated Macros".
11. In the section "Macros:" select the RESISTOR macro and press the << button.
12. Select the button Add Function
13. In the field "Name" of the "Function" dialog box, type: ACDC_DIODE" flag the checkbox Ignore Macro Selection Criteria and press the OK button.
14. Select the function: ACDC_DIODE
15. Select the item: " Associated Macros"
16. In the section "Macros:" Select the DIODE macro and press the << button
17. In the section "Macros:" Select the DIODE_INV macro and press the << button
18. Press the OK button to exit the Programming Environment dialog window.

The Software will prompt the following message: "**You have modified some code information in Macro Editor. Do you want that VIVA recreates your Test Program?**

Press the Yes button to exit the "Programming Environment" dialog window. The program will be recreated.

3.2.2 ADD A COMPLEX DEVICE

1. Double-click the icon on the Windows desktop to launch the VIVA Software
2. Login to the VIVA software as "Administrator" or "Developer" user category
3. In the top toolbar of the dialog window, press the

to enter the "Create Program" page.
4. In the Create Program page of the Wizard Environment, select the 

button.
5. In the "Assign Function" dialog window, tap the

button next to "Assign Complex to Selected Components" section. The " Complex Components Wizard" dialog window is prompted.

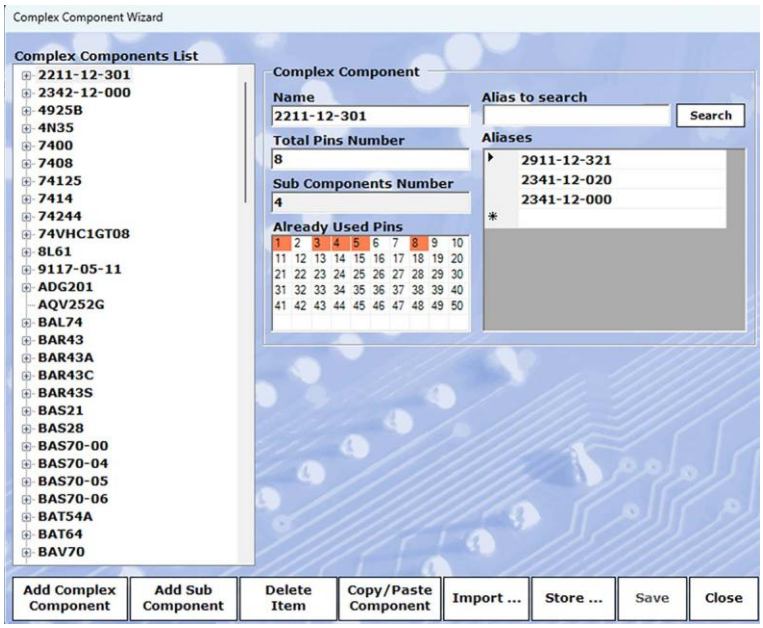


Fig. 3-2 VIVA SOFTWARE: The "Complex Component Wizard" dialog window

6. Press the button

at the bottom of the dialog window.

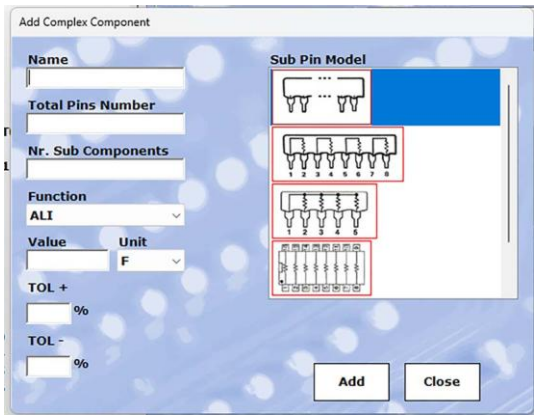


Fig. 3-3 VIVA SOFTWARE: "Add Complex Component" from Programming Environment

7. In the "Add Complex Component" dialog window, complete the fields as follows:

Field	Value
Name	ACDC
Total Pins Number	4
Nr. Sub components	3
Function (dropdown menu=	FACDC

8. Press the **Add** button and then the **Close** button to exit the "Add Complex Component" dialog window.

9. In the "Complex Component List" section of the "Complex Component Wizard" dialog window (see Fig. 3-2), select the complex component "ACDC".

10. Select: SUB1 and define the pins used for SUB1 (pin 1 and pin 2).

11. Select the function associated to SUB1: FACDC and press the **Save** button
12. Select: SUB2 and define the pins used for SUB2 (pin 1 and pin 2)

13. Select the function associated to SUB2: ACDC_DIODE and press the **Save** button
14. Select: SUB3 and define the pins used for SUB3 (pin 3 and pin 4)

15. Select the function associated to SUB3: ACDC_RESISTOR and press the **Save** button

16. Press the **Close** button to exit the " Complex Components Wizard" dialog window.

17. Press the **OK** in the "Assign Function" dialog window to exit.

Below it is reported the view of the "Complex Component Wizard" dialog window related to the three components of the complex device described in the example. This view also shows the values and tolerances used in the functions associated to the components of the complex device.

Complex Components list
.....
[-] ACDC
+------Sub1
+------Sub2
+------Sub3
[-] nnnn
.....

Used pins
1
2
0
0
0
0
0
0

Function
FACDC

☐ Value: 0 H
☐ TOL+: 0 %
☐ TOL-:0 %

Complex Components list
.....
[-] ACDC
+------Sub1
+------Sub2
+------Sub3
[-] nnnn
.....

Used pins
1
2
0
0
0
0
0
0

Function
ACDCD_DIODE

☑ Value: 0.75 V
☑ TOL+: 12 %
☑ TOL-:12 %

Complex Components list
.....
[-] ACDC
+------Sub1
+------Sub2
+------Sub3
[-] nnnn
.....

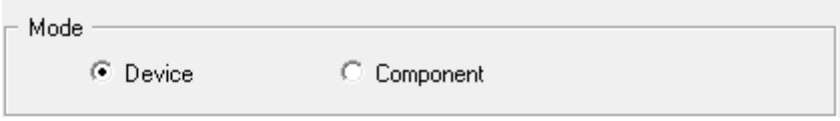
Used pins
3
4
0
0
0
0
0
0

Function
ACDCD_RESISTOR

☑ Value: 1200 O
☑ TOL+: 5 %
☑ TOL-:5 %

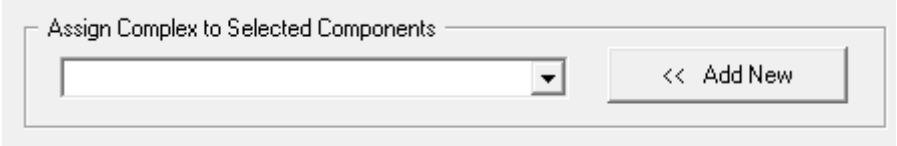
3.2.2.1 ASSIGN A DEVICE TO A COMPLEX DEVICE

1. Double-click the icon on the Windows desktop to launch the VIVA Software
2. Login to the VIVA software as "Administrator" or "Developer" user category
3. In the top toolbar of the dialog window, press the Create to enter the "Create Program" page.
4. In the Create Program page of the Wizard Environment, select the Assign Function button.
5. In the "Mode" section of the "Assign Function" dialog window, select the "Device" radio button



XDIODERESISTORS

In the "Assign Complex to Selected Components" section, use the dropdown menu to **select the complex device** (e.g. ***ACDC***) to be associated to the selected item. (e.g. X*DIODERESISTORS*).



6. Press the Apply button to apply the changes: (see example "Device Before" and "Device After")
7. Press the OK button to exit the "Assign Function" dialog window. The program will be recreated.

If an existing device has been associated to a complex device, the existing device is renamed as the complex device, as shown in the view "Device After".

Device Before

Device	**Function**	**Components**
.....	.....	.....
XDIODERESISTORS	UNKNOWN	ACDC1,ACDC2
DIODES	DIODE	DI1,DI2
.....	.....	.....

Device After

Device	**Function**	**Components**
.....	.....	.....
ACDC	<Complex>	ACDC1,ACDC2
DIODES	DIODE	DI1,DI2
.....	.....	.....

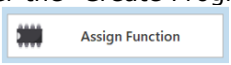
After recreating the program, all the macros belonging to the functions of the "Sub Components" that make up the complex device will be associated to the device.
The "Debug Environment" dialog window will show the sections[6] where the macros associated to the complex components will be executed.

```
Debug Environment
...
- FACDC
  +----- - ACDC1
  |          +--   DIODE : 1-2
  |          +-- DIODE_INV : 1-2
  |          +--   RESISTOR : 3-4
  +----- - ACDC2
             +--   DIODE : 1-2
             +--   DIODE_INV : 1-2
             +--   RESISTOR : 3-4
```

[6] The name of the section is the name of the function associated to the first "sub device" of the complex device. In our example, the "SUB1" component has been created, along with the empty function "FACDC", to achieve an increased program readability (i.e to have a section which identifies the complex device and the system macros executed to test the components belonging to the complex device).

3.2.2.2 ASSIGN A COMPONENT TO A COMPLEX DEVICE

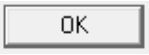
1. Double-click the icon on the Windows desktop to launch the VIVA Software
2. Login to the VIVA software as "Administrator" or "Developer" user category
3. In the top toolbar of the dialog window, press the

to enter the "Create Program" page.
4. In the Create Program page of the Wizard Environment, select the 

button.
5. In the "Mode" section of the "Assign Function" dialog window, select the "Component" radio button

6. In the section: "Assign Complex to Selected Components", use the dropdown menu to select the complex device (e.g.. ACDC) to be associated to the selected item. (e.g.. ACDC1 or ACDC2).

7. Press the

button to apply the changes: (see example "Component Before" and "Component After")
8. Press the 

button to exit the "Assign Function" dialog window. The program will be recreated.

If a component of an existing device is associated to a complex device, after assignment the component will belong to the complex device, as shown in the example "Component After".

Component Before:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	UNKNOWN	XDIODERESISTORS		1e+005	...	4
ACDC2	UNKNOWN	XDIODERESISTORS		1e+005	...	4
.....	.....	.....	.....	.....	...	.....

Component After:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	<Complex>	**ACDC**		1e+005	...	4
ACDC2	UNKNOWN	XDIODERESISTORS		1e+005	...	4
.....	.....	.....	.....	.....	...	.....

After recreating the program, all the macros belonging to the functions of the "Sub Components" that make up the complex device will be associated to the component.
The "Debug Environment" dialog window will show the sections[7] where the macros associated to the complex components will be executed.

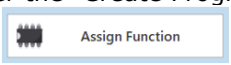
```
Debug Environment
(if assignment has been made only for ACDC1
  ...
 [-] FACDC
   +----[-] ACDC1
   |            +--      DIODE : 1-2
   |            +-- DIODE_INV : 1-2
   |            +--      RESISTOR : 3-4
  ....
```

```
Debug Environment
(if assignment  has been made for ACDC1 and ACDC2
  ...
 [-] FACDC
   +----[-] ACDC1
   |            +--      DIODE : 1-2
   |            +-- DIODE_INV : 1-2
   |            +--      RESISTOR : 3-4
   +----[-] ACDC2
                +--      DIODE : 1-2
                +--      DIODE_INV : 1-2
                +--      RESISTOR : 3-4
```

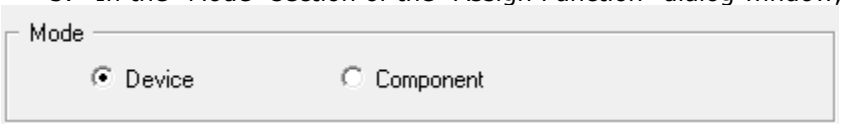
[7] The name of the section is the name of the function associated to the first "sub device" of the complex device. In our example, the "SUB1" component has been created, along with the empty function "FACDC", in order to increase the program readability, i.e to have a section which identifies the complex device and the system macros executed to the components belonging to the complex device.

3.2.2.3 ASSIGN A FUNCTION TO A COMPONENT

1. Double-click the icon on the Windows desktop to launch the VIVA Software
2. Login to the VIVA software as "Administrator" or "Developer" user category
3. In the top toolbar of the dialog window, press the

to enter the "Create Program" page.
4. In the Create Program page of the Wizard Environment, select the 

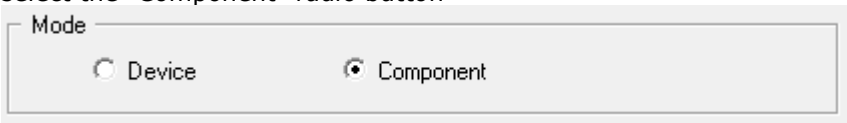
button.
5. In the "Mode" section of the "Assign Function" dialog window, select the "Device" radio button



and add "***DRelais"*** *device*

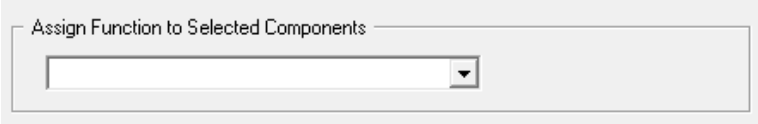
<u>Or:</u>

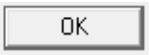
select the "Component" radio button



and select the component **"RL1"** or **"U2"**

6. In the field "Assign Function to Selected Components" use the dropdown menu to select the function to be associated to the component. (e.g. RELAY or AUTIC)



7. Press the 

button to exit the "Assign Function" dialog window. The program will be recreated.

After the new program creation, the "Debug Environment" will show:

```
Debug Environment
......
[-] RELAIS
 +------[-] RL1
            +--RELAIS
```

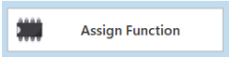
o

```
Debug Environment
......
[-] AUTIC
 +------[-] U2
            +--------AUTIC:5
            +--------AUTIC:6
            +--------AUTIC:8
            +--------AUTIC:i
```

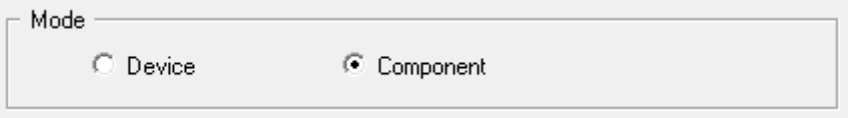
3.2.2.4 ASSIGN A MACRO TO A COMPONENT

This paragraph describes the basic steps to assign a macro to a component of the board. This is an **alternative way to check a complex component**. After generating the program, it may be required to access the Debug Environment to associate the pins of the component to be checked to the reference pins included in the created program.

1. Double-click the icon on the Windows desktop to launch the VIVA Software
2. Login to the VIVA software as "Administrator" or "Developer" user category
3. In the top toolbar of the dialog window, press the

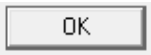
to enter the "Create Program" page.
4. In the Create Program page of the Wizard Environment, select the 

button.
5. In the "Mode" section of the "Assign Function" dialog window, select the "Component" radio button



6. In section: "Assign Macro to Selected Components": select the required macro to check the selected component. (e.g. NEW_RESISTOR)

7. Press the

button to apply the changes (see example: "Component Before" and "Component After"
8. Press the 

button to exit the "Assign Function" dialog window. The program will be recreated.

If a macro has been associated to a component, then the component is still associated to the previous function, but the macro executed is indicated in field "ForcedMacro", as shown in the view: "Component After".

Component Before:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	<Complex>	DACDC		0.7		4
ACDC2	UNKNOWN	XDIODERESISTORS		0.7		4
.....	.....	.....	.....	.....	...	.....

Component After:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	<Complex>	DACDC		0.7		4
ACDC2	UNKNOWN	XDIODERESISTORS	**NEW_RESISTOR**	0.7		4
.....	.....	.....	.....	.....	...	.....

If the macro “NEW_RESISTOR” has been developed on purpose for the complex component, it may be required to access the Debug Environment to associate the pins of the component to be checked to the reference pins included in the test program. In the Debug Environment it is possible to view the section where the macro associated to complex components is executed.

```
Debug Environment
  ...
[-] FACDC
   +----[-] ACDC1
   |          +--  DIODE : 1-2
   |          +--  DIODE_INV : 1-2
   |          +--  RESISTOR : 3-4
  ....
[-] UNKNOWN
   +----[-] ACDC2
   |          +--  NEW_RESISTOR
  ....
```

3.2.3 CHANGE THE MACROS BELONGING TO A FUNCTION

This section describes the steps required to change the system macros that make up an existing function. Specifically, the "Diode" function available with the VIVA Software, contains a single macro which performs the functional test of direct polarity "Diode" kind components. The following steps explain how to change the "Diode" function to test also the reversed polarity "Diode" kind components. In other words, the "Diode" function will contain the "DIODE" and "DIODE_INV" macros.

1. Double-click the icon on the Windows desktop to launch the VIVA Software.
2. Login to the VIVA software as "Administrator" or "Developer" user category
3. In the main page of the VIVA software, press the button to open the right-hand side toolbar.
4. In the Utilities section, press the FAT Markers_ori Version 1 to enter the Program Manager Utility.
5. In the Program Manager Utility, use the search bar or the side scroll bar until you find the and enter the Programming Environment.

<u>In the tab "General" of the Programming Environment dialog window:</u>

6. Select the function: "Diode" and select the item " Associated Macros"
7. In the section "Macros" , select the macro: "DIODE_INV" and press the << button.

The Programming Environment will show the architecture of the "Diode" function, before and after the changes.

<u>BEFORE</u>

```
[-] Functions
 +------[-] Diode
        +----[-] Asscociated Macros
        |          +--DIODE
        +----[-] Patterns
                   +--##CR*
                 +--##D*
                 +--##GR*
                 +--##LD*
                 +--##LED*
                 +--DIODE*
```

<u>AFTER</u>

```
[-] Functions
 +------[-] Diode
        +----[-] Asscociated Macros
        |          +--DIODE
        |          +--DIODE_INV
        +----[-] Patterns
                   +--##CR*
                   +--##D*
                   +--##GR*
                   +--##LD*
                   +--##LED*
                   +--DIODE*
```

As an alternative, it is possible to define a new function which contains the "DIODE" and "DIODE_INV", macros, without changing the existing "Diode" function.

3.3 COMPLEX DEVICE CONSISTING OF ICS

The operational amplifier has been used as an example of a digital complex component.

The operational amplifier is a high gain DC coupled differential amplifier. It can be used in several applications: non-inverting amplifier, inverting amplifier, differential amplifier, buffer, comparator, oscillator, active filter, etc.
The VIVA Software makes three system macros available to check the main operating conditions of an operational amplifier:

- Comparator (COMPARATOR)
- Operational amplifier (OP_AMP_VF)
- Operational amplifier gain (OP_AMP_GAIN)

The "OpAmp" function available in the Programming Environment of the VIVA Software, schedules the execution of three macros on a sample device made up by an operational amplifier with output on pin 3 and inputs on pin 1 and 2. Normally, in a "in-circuit" context, the operational amplifier may be checked only in one of the three scheduled modes, according to the circuitry connections to the components to be tested. Besides, the same IC can contain more instances for the device to be checked and, most likely, the real component pins other than pins 1, 2 and 3 of the sample.

The reference UUT used as a sample schedules four ICs, with brand name LF412, featuring two operational amplifiers each. In the UUT CAD data, the brand name has been assigned to field "Device" and the digital components are identified by the alphanumeric string: "U*ii*", where "*ii*" is the identification number for the digital components mounted on the UUT.

In the "Programming Environment" of the VIVA Software, the digital components used for the example, should be checked with the following features:

U44 belonging to typology or device "LM7812M" can be checked with the "OpAmp" function according to directory "Patterns" associated to the "OpAmp" function in the Programming Environment.
U33 belonging to typology or device "LF412" can be checked with "Digital" function according to directory "Patterns" associated to function "Digital" in the Programming Environment.
U34 belonging to typology or device "LF412" can be checked with "Digital" function according to directory "Patterns" associated to function "Digital" in the Programming Environment.
U35 belonging to typology or device "LF412" can be checked with "Digital" function according to directory "Patterns" associated to function "Digital" in the Programming Environment.
U36 belonging to typology or device "LF412" can be checked with "Digital" function according to directory "Patterns" associated to function "Digital" in the Programming Environment".

On the ground of the real components and their "in-circuit" context, the features to check the components are the following:

U44 belonging to typology or device "LM7812M" can be checked with "OpAmp" function according to directory "Patterns" associated to the "OpAmp" function in the Programming Environment.
U33 belonging to typology or complex device "DLF412_AMPL" can be checked with the new ""FOP_AMPL" function which schedules the execution of the "OP_AMP_GAIN" macro on the sub devices of the complex device
U34 belonging to typology or complex device"DLF412_CMP" can be checked with the new "FOP_CMP" function, which schedules the execution of the "COMPARATOR" macro on the sub devices of the complex component.
U35 belonging to typology or complex device "DLF412_VF" can be checked with the new "FOP_VF" function which schedules the execution of the "OP_AMP_VF" macro on the sub devices of the complex component.
U36 belonging to typology or device "DLF412" can be checked with "OpAmp" function which schedules the execution of the three macros: "COMPARATOR", "OP_AMP_VF" and OP_AMP_GAIN on the sub devices of the complex component.

Therefore, it is necessary to define three functions in order to check the three possible "in_circuit" locations of the components, and a possible empty function in order to improve the program readability:

FOP_AMPL	function to check the operational amplifier used as signal amplifier.
FOP_CMP	function to check the operational amplifier used as signal comparator.
FOP_VF	function to check the operational amplifier used as buffer.
FOP_LF412	empty function to improve the test program readability, grouping in one section all the tests executed on all LF412 kind components.

In order to solve pin mapping problems between real and sample components, it is necessary to define a series of complex devices, which allow to define the pin mapping and the function used to check the sub devices.

DLF412_AMPL	complex device which groups the LF412 components which can be checked as amplifiers.
DLF412_CMP	complex device which groups the LF412 components which can be checked as comparators.
DLF412_VF	complex device which groups the LF412 components which can be checked as buffers.
DLF412	complex device which groups the LF412 components which can be checked as amplifiers, comparators and buffers.

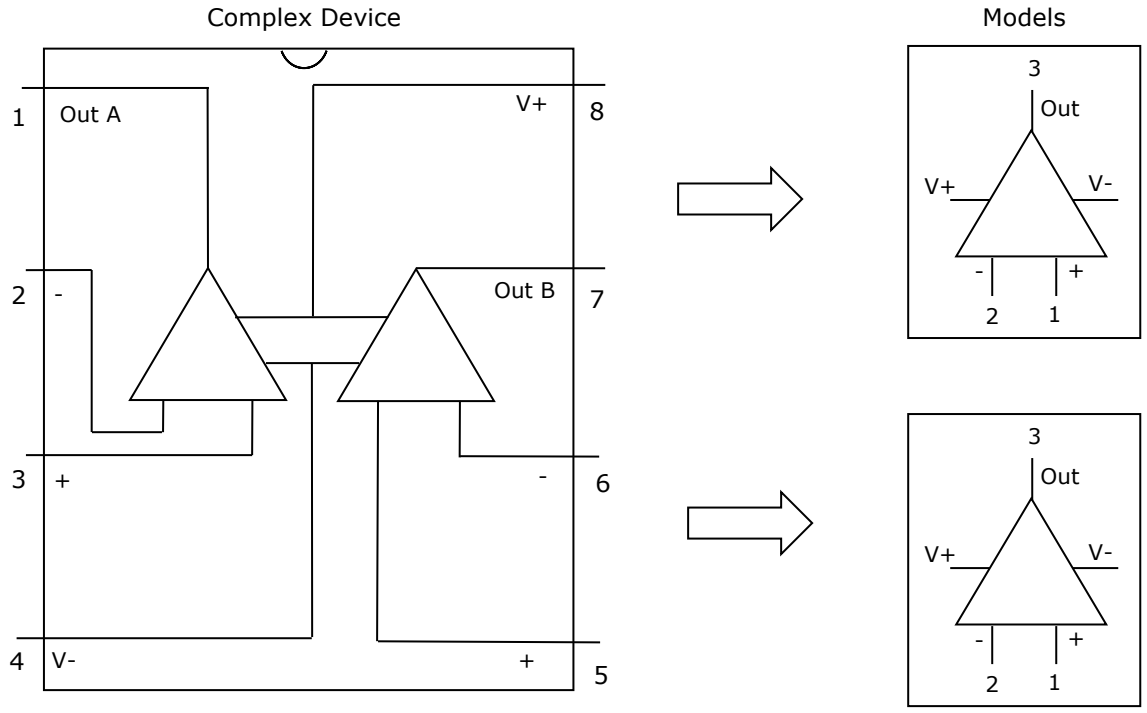


Fig. 3-4 VIVA SOFTWARE: Complex device and VIVA models for Operational Amplifier

To name the new functions and complex devices generated, the prefix "F" and "D" have been used, respectively to clearly identify the generated structures.

The drawings of Fig. 3-5, show a sample of the data displayed in the "Add Item Wizard"[8] dialog window, which allows to view, some reference data used in the next pages of this manual. In brackets you find the function used to test the device operation in the VIVA Environment.

"Add Item Wizard" after "Import Data":

```
Device board list
.....
LF411          ( Digital )
LF412          ( Digital )
.....
LM7812M        ( OpAmp )*
.....
```

```
Component board list
.....
U33
U34
U35
U36
U37
.....
```

Fig. 3-5 VIVA SOFTWARE: Device and Component list after "import Data"

The functions associated to the device are defined by item "Patterns" included in the description of "OpAmp" and "Digital" functions of the Programming Environment as explained in Fig. 3-6.

"OpAmp" and "Digital" functions in the Debug Environment

```
Function: OpAmp

[-] Functions
 +------[-] OpAmp
        +----[-] Asscociated Macros
        |        +-----COMPARATOR
        |        +-----OP_AMP_VF
        |        +-----OP_AMP_GAIN
        +----[-] Patterns
               +--------LM*
```

```
Function: Digital

[-] Functions
 +------[-] Digital
        +----[-] Asscociated Macros
        |        +---CLUSTER
        +----[-] Patterns
                 +----##U*
                 +----##IC*
                 +----##74*
                 +----##52*
                 +----##OQ*
                 +----##MC*
                 +----##QFP*
                 +----##DIGITAL*
```

Fig. 3-6 VIVA SOFTWARE: "OpAmp" and "Digital" functions view from the Debug Environment

[8] To access the "Add Items Wizard" dialog window, press the "Add Pins …", button in the "Edit Data" page of the Wizard Environment.

The Fig. 3-7 shows the data available in the "Assign Function" dialog window, which contains the directories used to create the test program.

The Fig. 3-8 shows the grid of sections, macros and functions in the Debug Environment to test the component.

"Assign Function – Device" before "Create Program":

Device	Function	Components
.....	.....	.....
LF411	DIGITAL	U37
LF412	DIGITAL	U33, U34, U35, U36
.....	.....	.....
LM7812M	OPAMP	U44
.....	.....	.....
.....	.....	.....

"Assign Function – Component" before "Create Program":

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	...
U33	DIGITAL	LF412	.....	.....	... 8	
U34	DIGITAL	LF412	.....	.....	... 8	
U35	DIGITAL	LF412	.....	.....	... 8	
U36	DIGITAL	LF412	.....	.....	... 8	
U37	DIGITAL	LF411	.....	.....	... 8	
.....	.....	.....	.....	.....	...	...
U44	OPAMP	LM7812M	.....	.....	...	8
.....	.....	.....	.....	.....	...	...

Fig. 3-7 VIVA SOFTWARE: "Assign Function" tables

Debug Environment after "Create Program":

```
Debug Environment
  ...
[-] OPAMP
  +----[-] U44
            +-- COMPARATOR
            +-- OP_AMP_VF
            +-- OP_AMP_GAIN
  ...
[-] DIGITAL
  +----[-] U37
            +-- CLUSTER
  ....
  +----[-] U33
            +-- CLUSTER
  +----[-] U34
            +-- CLUSTER
  +----[-] U35
            +-- CLUSTER
  +----[-] U36
            +-- CLUSTER
  ....
```

Fig. 3-8 VIVA SOFTWARE: "Debug Environment" of the VIVA Software

3.3.1 ADD FUNCTIONS

To check the complex components described in the example, it is necessary to create four new functions in the Programming Environment. The structure of these functions is shown in Fig. 3-9

"Programming Environment" after creating the functions for complex devices":

```
Function: FOP_LF412

- Functions
 +------- - FOP_LF412
          +---- - Asscociated Macros
          |
          +---- - Patterns
          ....
```

```
Function: FOP_AMPL

- Functions
 +------- - FOP_AMPL
          +---- - Asscociated Macros
          |     +--------OP_AMP_GAIN
          +---- - Patterns
          ....
```

```
Function: FOP_CMP

- Functions
 +------- - FOP_CMP
          +---- - Asscociated Macros
          |     +--------COMPARATOR
          +---- - Patterns
          ....
```

```
Function: FOP_VF

- Functions
 +------- - FOP_VF
          +---- - Asscociated Macros
          |     +--------OP_AMP_VF
          +---- - Patterns
          ....
```

Fig. 3-9 VIVA SOFTWARE: Adding functions in the Programming Environment

3.3.2 ADD A COMPLEX DEVICE

The Fig. 3-10, Fig. 3-11 and Fig. 3-12 reproduce the structure of the three complex devices added to UUT database used as an example:

- DLF412_AMPL
- DLF412_CMP
- DLF412_VF

Complex Components list
.....
- DLF412_AMPL
+------Sub1
+------Sub2
+------Sub3
- nnnn
.....

Used pins
0
0
0
0
0
0
0

Function
LF412

☐ Value: 0 H
☐ TOL+: 0 %
☐ TOL-: 0 %

Complex Components list
.....
- DLF412_AMPL
+------Sub1
+------Sub2
+------Sub3
- nnnn
.....

Used pins
3
2
1
0
0
0
0

Function
FOP_AMPL

☑ Value: 150 Nr
☑ TOL+: 10 %
☑ TOL-: 10 %

Complex Components list
.....
- DLF412_AMPL
+------Sub1
+------Sub2
+------Sub3
- nnnn
.....

Used pins
5
6
7
0
0
0
0

Function
FOP_AMPL

☑ Value: 150 Nr
☑ TOL+: 10 %
☑ TOL-: 10 %

Fig. 3-10 VIVA SOFTWARE: Complex device "Operational Amplifier" used as Amplifier

Complex Components list
.....
- DLF412_CMP
+------Sub1
+------Sub2
+------Sub3
- nnnn
.....

Used pins
0
0
0
0
0
0
0

Function
FOP_LF412

☐ Value: 0 H
☐ TOL+: 0 %
☐ TOL-: 0 %

Complex Components list
.....
- DLF412_CMP
+------Sub1
+------Sub2
+------Sub3
- nnnn
.....

Used pins
3
2
1
0
0
0
0

Function
FOP_CMP

☑ Value: 150 Nr
☑ TOL+: 10 %
☑ TOL-: 10 %

Complex Components list
.....
- DLF412_CMP
+------Sub1
+------Sub2
+------Sub3
- nnnn
.....

Used pins
5
6
7
0
0
0
0

Function
FOP_CMP

☑ Value: 150 Nr
☑ TOL+: 10 %
☑ TOL-: 10 %

Fig. 3-11 VIVA SOFTWARE: Complex device "Operational Amplifier" used as Comparator

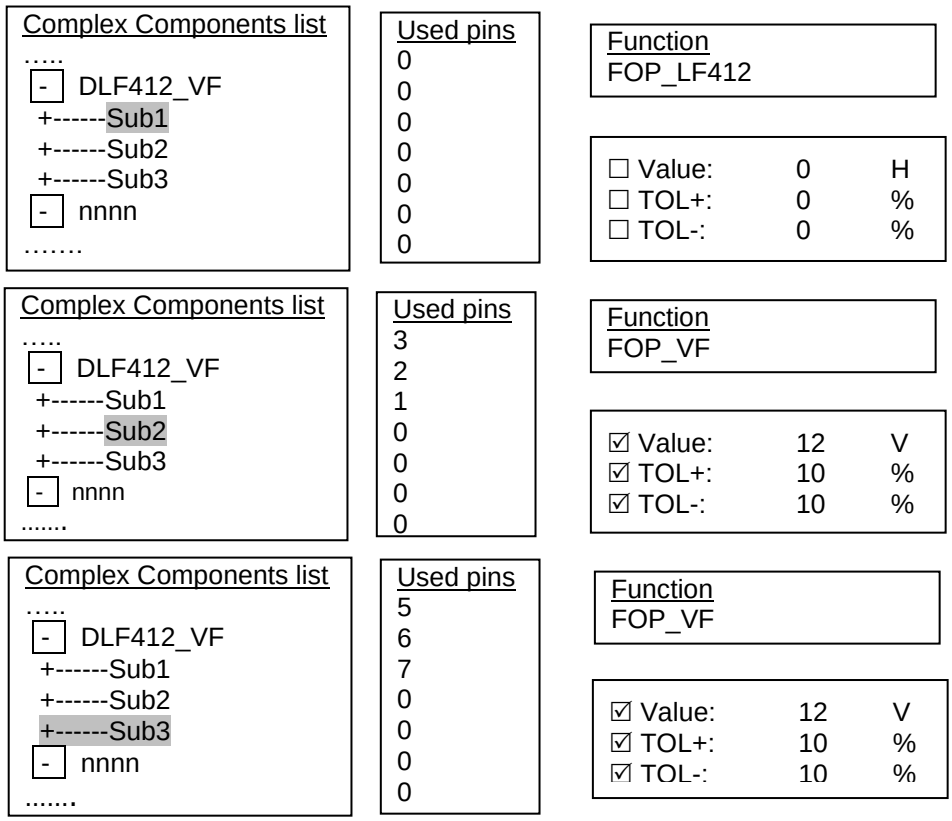


Fig. 3-12 VIVA SOFTWARE: Complex device "Operational Amplifier" used as Voltage Follower

3.3.3 ASSIGN COMPONENTS TO COMPLEX DEVICES

The Fig. 3-13, reproduces the data viewed in the "Assign Function" dialog window, which contains the directory used to create the test program, after the following associations:

- U33: associated to DLF412_AMPL complex device
- U34: associated to DLF412_CMP complex device
- U35: associated to DLF412_VF complex device

The Fig. 3-14, reproduces the view obtained in the "Add Item Wizard" dialog window, which outlines the changes in the UUT components and devices list after the components have been associated to complex devices.

The Fig. 3-15, reproduces the view of the checks executed obtained in the Debug Environment, after assigning components to complex devices and recreating the test program.

"Assign Function – Component" after "Assign Function":

Component	**Function**	**Device**	**ForcedMacro**	**Value1**	**...**	**TotPinNum**
……..	……	…….	…….	…….	…	…
U33	<Complex>	DLF412_AMPL	…….	…….	…	8
U34	<Complex>	DLF412_CMP	…….	…….	…	8
U35	<Complex>	DLF412_VF	…….	…….	…	8
U36	DIGITAL	LF412	…….	…….	…	8
U37	DIGITAL	LF411	…….	…….	…	8
……..	……	…….	…….	…….	…	..
U44	OPAMP	LM7812M	…….	…….	…	8
……..	……	…….	…….	…….	…	..

Fig. 3-13 VIVA SOFTWARE: "Assign Function" Component table

"Add Item Wizard" dialog window after "Assign Function":

Device board list
…..
DLF412_AMPL (FOP_LF412)
DLF412_CMP (FOP_LF412)
DLF412_VF (FOP_LF412)
…..
FOP_AMPL (FOP_AMPL)
FOP_CMP (FOP_CMP)
FOP_LF412 (FOP_LF412)
FOP_VF (FOP_VF)
…..
LF411 (Digital)
LF412 (Digital)
…..
LM7812M (OpAmp)
…..

Component board list
…..
U33
U33#1
U33#2
U33#3
U34
U34#1
U34#2
U34#3
U35
U35#1
U35#2
U35#3
U36
U37
…..

Fig. 3-14 VIVA SOFTWARE: Device and Component list after "Assign Function"

"Debug Environment" grid after "Create Program":

```
Debug Environment

  ...
[-] OPAMP
  +----[-] U44
           +--COMPARATOR
           +--OP_AMP_VF
           +--OP_AMP_GAIN
  ...
[-] DIGITAL
  +----[-] U37
           +--CLUSTER
  +----[-] U33
           +--CLUSTER
  ....

[+] END

[-] FOP_LF412
  +----[-] U33
           +--OP_AMP_GAIN : 3-2-1
           +--OP_AMP_GAIN : 5-6-7
  +----[-] U34
           +--COMPARATOR : 3-2-1
           +--COMPARATOR : 5-6-7
  +----[-] U35
           +--OP_AMP_VF : 3-2-1
           +--OP_AMP_VF : 5-6-7
```

Fig. 3-15 VIVA SOFTWARE: "Debug Environment"

3.3.4 COMPLEX DEVICE MADE UP BY SEVERAL MACROS

In order to complete the example described above, this chapter describes the data viewed in the different Environments of the VIVA software, in regards to the addition of a complex device (DLF412). The "OpAmp" function is used to test the sub devices. This function uses the three macros available to check the operational amplifier kind components.

The component U36 is associated to "DLF412" complex device, and the drawings reproduced below show the new data added to those viewed previously.

"Debug Environment": "OpAmp" function used for "DLF412" complex device:

```
Function: OpAmp

[-] Functions
 +------[-] OpAmp
        +----[-] Asscociated Macros
              +--------COMPARATOR
        |     +--------OP_AMP_VF
              +--------OP_AMP_GAIN
```

Create a complex device:

- DLF412

Complex device: DLF412

```
Complex Components list
.....
[-] DLF412
 +------Sub1
 +------Sub2
[-] nnnn
.......
```

Used pins
3
2
1
0
0
0
0

Function
OpAmp

☐ Value: 0 H
☐ TOL+: 0 %
☐ TOL-: 0 %

```
Complex Components list
.....
[-] DLF412
 +------Sub1
 +------Sub2
[-] nnnn
.......
```

Used pins
5
6
7
0
0
0
0

Function
OpAmp

☐ Value: 150 H
☐ TOL+: 10 %
☐ TOL-: 10 %

"Assign Function – Component" after "Assign Function":

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	...
U33	<Complex>	DLF412_AMPL	.....	.....	...	8
U34	<Complex>	DLF412_CMP	.....	.....	...	8
U35	<Complex>	DLF412_VF	.....	.....	...	8
U36	<Complex>	DLF412	.....	.....	...	8
U37	DIGITAL	LF411	.....	.....	...	8
.....	.....	.....	.....	.....	...	..
U44	OPAMP	LM7812M	.....	.....	...	8
.....	.....	.....	.....	.....	...	..

"Add Item Wizard" dialog window after "Assign Function":

Device board list

```
.....
DLF412              ( OpAmp )
DLF412_AMPL         ( FOP_LF412 )
DLF412_CMP          ( FOP_LF412 )
DLF412_VF           ( FOP_LF412 )
.....
FOP_AMPL            ( FOP_AMPL )
FOP_CMP             ( FOP_CMP )
FOP_LF412           ( FOP_LF412 )
FOP_VF              ( FOP_VF )
.....
LF411          ( Digital )
LF412          ( Digital )
.....
LM7812M        ( OpAmp )
.....
```

Component board list

```
.....
U33
U33#1
U33#2
U33#3
U34
U34#1
U34#2
U34#3
U35
U35#1
U35#2
U35#3
U36
U36#1
U36#2
U37
.....
```

"Debug Environment" grid after "Create Program":

Debug Environment

```
...
[-] OPAMP
  +----[-] U44
           +--COMPARATOR
           +--OP_AMP_VF
           +--OP_AMP_GAIN
  +----[-] U36
           +--COMPARATOR : 3-2-1
           +--OP_AMP_VF : 3-2-1
           +--OP_AMP_GAIN : 3-2-1
           +--COMPARATOR : 5-6-7
           +--OP_AMP_VF : 5-6-7
           +--OP_AMP_GAIN : 5-6-7
...

[-] DIGITAL
  +----[-] U37
           +--CLUSTER
....

[+] END

[-] FOP_LF412
  +----[-] U33
           +--OP_AMP_GAIN : 3-2-1
           +--OP_AMP_GAIN : 5-6-7
  +----[-] U34
           +--COMPARATOR : 3-2-1
           +--COMPARATOR : 5-6-7
  +----[-] U35
           +--OP_AMP_VF : 3-2-1
           +--OP_AMP_VF : 5-6-7
```

SEICA SpA
via Kennedy 24
10019 Strambino - TO
ITALY

Tel: +39 0125 6368.11
Fax: +39 0125 6368.99

e-mail: sales@seica.com